

ML: TASK-03

Machine Learning Classifiers on Iris Species Data

Comparative Analysis of SVM, KNN, and Random Forest

1. Introduction

Understanding of the additional classification algorithm chosen for this task — **Random Forest** — along with a comparative analysis of all three classifiers implemented: SVM, KNN, and Random Forest. The Iris dataset was used as the benchmark for this study. It contains 150 samples across three species of iris flowers (Iris setosa, Iris versicolor, Iris virginica) with four measurement features: sepal length, sepal width, petal length, and petal width.

2. Random Forest — Conceptual Explanation

2.1 How the Algorithm Works

Random Forest is an ensemble learning method built on top of Decision Trees. The core idea is straightforward: instead of relying on a single decision tree (which tends to overfit), we build many trees — each one slightly different — and let them vote on the final prediction.

The training process follows these steps:

- **Bootstrap Sampling:** For each tree, a random subset of the training data is selected with replacement (this is called bootstrap sampling). On average, each tree sees about 63% of the original training data; the rest is left out and can be used for internal validation.
 - **Random Feature Selection:** At each node split within a tree, only a random subset of features is considered rather than all features. This introduces diversity among the trees and prevents all of them from learning the same dominant features.
 - **Tree Growth:** Each tree is grown to its full depth with no pruning. Individual trees have high variance (they overfit their bootstrap sample), but combining many of them averages out the
-

individual errors.

- **Majority Voting:** For classification, each tree casts a vote for a class label. The class that receives the most votes across all trees becomes the final prediction.

2.2 Key Assumptions

- The individual base trees should have some predictive power (better than random guessing).
- The errors made by individual trees should be somewhat independent — this is achieved through bootstrap sampling and random feature selection.
- A sufficient number of trees must be used to allow the law of large numbers to reduce variance; typically 100–500 trees is adequate for most problems.
- The dataset should not have excessive class imbalance, as majority voting can be biased toward the dominant class.

2.3 Strengths

- Highly accurate and robust — consistently one of the best off-the-shelf classifiers.
- Handles both numerical and categorical features well.
- Resistant to overfitting due to averaging across many trees.
- Provides feature importance scores that help understand which inputs drive predictions.
- Requires relatively little hyperparameter tuning — works well with default settings.
- Can handle missing values and noisy data better than single decision trees.
- Naturally handles multiclass classification problems without modification.
- No requirement for feature scaling — unaffected by the magnitude of features.

2.4 Limitations

- Less interpretable than a single decision tree — it is harder to visualize or explain the logic behind a prediction when 100+ trees are involved.
- Computationally heavier in terms of memory and inference time, especially with very large numbers of trees or high-dimensional data.
- Can be biased toward features with more categories or higher cardinality in certain splitting criteria.
- Prediction latency is higher compared to simple models like Logistic Regression or KNN on small datasets, making it less suitable for real-time applications with strict latency requirements.
- Does not extrapolate well beyond the range of the training data — predictions are bounded by the observed training values.

2.5 When Random Forest Performs Well

Random Forest is a strong choice in the following scenarios:

- Medium to large structured/tabular datasets where feature interactions are complex.
-

- When the dataset has both relevant and irrelevant features — RF naturally de-emphasizes weak predictors through feature selection at each split.
- When a quick, high-accuracy baseline is needed without much tuning.
- When feature importance analysis is required alongside predictions.
- Multi-class classification problems like the Iris dataset.
- When training time is not severely constrained.

2.6 When Random Forest May Not Be Suitable

There are situations where Random Forest may not be the best option:

- Very small datasets (like Iris with 150 samples) — simpler models may generalize just as well with less overhead.
- Real-time prediction systems where low latency is critical — RF is slower to predict than SVM or logistic regression.
- When full model transparency and legal interpretability are required (e.g., in regulated industries like healthcare or finance).
- Streaming or online learning scenarios — RF requires the full dataset to train and does not update incrementally.
- Extremely high-dimensional sparse data like text (tf-idf features) — linear models or gradient boosting often outperform RF in this domain.

3. Comparative Analysis — SVM vs KNN vs Random Forest

All three algorithms were evaluated on the Iris dataset after standard preprocessing (dropping the Id column, label encoding, 80/20 train-test split, StandardScaler). Below is a summary comparison of each algorithm across key dimensions:

Dimension	SVM	KNN	Random Forest
Core Idea	Find max-margin hyperplane between classes	Majority vote of K nearest training samples	Majority vote of 100 decision trees (bagging)
Training Speed	Moderate (depends on kernel)	Instant (no training phase)	Moderate (100 trees)
Prediction Speed	Fast	Slow for large datasets (distance to all points)	Moderate (100 tree traversals)
Feature Scaling Needed?	Yes — sensitive to feature magnitude	Yes — distance-based, needs equal scale	No — tree splits are scale-invariant
Multiclass Handling	One-vs-One by default (handled internally)	Natural — majority vote among K neighbors	Natural — majority vote across all trees
Interpretability	Low — margin-based boundary is abstract	High — neighbors can be shown directly	Low — many trees make it a black box
Feature Importance	Not directly available	Not available	Available — built-in gini importance
Overfitting Risk	Low — maximizes margin	Low with proper K (high at K=1)	Low — averaging reduces variance
Performance on Iris	96–100% (Linear, RBF best)	~96–97% (K=5)	~96–100% (100 trees)

3.1 Performance on Iris Dataset

The Iris dataset is a favorable benchmark for all three algorithms because it is clean, balanced (50 samples per class), and has well-separated clusters — particularly Iris setosa, which is almost perfectly linearly separable from the other two species. The main challenge is the overlap between Iris versicolor and Iris virginica, which share similar sepal and petal measurement ranges.

SVM with Linear and RBF kernels exploits this separability very effectively — the maximum-margin boundary places itself in the gap between clusters with minimal misclassifications. Random Forest handles the versicolor-virginica overlap through ensemble averaging, which smooths out the noisy boundaries that individual trees might draw. KNN benefits from the compact clustering — nearby neighbors in this dataset are almost always from the correct class.

4. Key Learnings and Interpretation of Results

4.1 Feature Importance Insight

One of the most valuable outputs from the Random Forest model was the feature importance analysis. The results confirmed that `PetalLengthCm` and `PetalWidthCm` are by far the most discriminative features — contributing roughly 85–90% of the total feature importance. `SepalLengthCm` and `SepalWidthCm` contribute far less. This aligns with what botanists have long known: petal dimensions vary dramatically across Iris species, while sepal dimensions have more overlap.

This insight has a practical implication — if we needed to reduce measurement effort in a real-world scenario (e.g., a field classification tool), we could likely achieve near-identical accuracy using only petal measurements.

4.2 The Importance of Feature Scaling

SVM and KNN both rely on distance calculations, so features with larger absolute values would dominate if not scaled. For example, `PetalLengthCm` ranges from about 1 to 7 cm while `SepalWidthCm` ranges from about 2 to 4.4 cm. Without standardization, the petal length would artificially dominate the distance metric. Applying `StandardScaler` ensured all features contributed equally to the model training. Random Forest, on the other hand, is completely unaffected by scaling since tree splits are based on threshold comparisons rather than distance calculations.

4.3 Kernel Selection in SVM

The kernel choice had a clear impact on SVM performance. The linear and RBF kernels performed best, which is consistent with the known structure of the Iris dataset. This reinforces an important practical rule: always try the RBF kernel as a default for SVM on tabular data, and also compare against linear — often one of these two will give the best result without needing more exotic kernels like polynomial or sigmoid.

4.4 Dataset Size and Algorithm Choice

The Iris dataset has only 150 samples, which means differences between algorithms are relatively small and all methods perform well. In larger, noisier real-world datasets, the advantages of ensemble methods like Random Forest become more pronounced — the variance-reduction effect of averaging many trees becomes significant when individual trees might otherwise overfit. SVM also scales well to larger datasets (with appropriate solvers), while KNN becomes computationally expensive as the training set grows since every prediction requires a full search through all training points.

4.5 Multiclass Classification

All three algorithms handled the three-class problem well. SVM internally decomposes multiclass problems into multiple binary classifiers (One-vs-One strategy), KNN naturally handles it through majority voting among K neighbors, and Random Forest handles it through voting across trees — each tree itself capable of outputting one of three classes. This task reinforced that for balanced multiclass problems with clean data, the choice of algorithm matters less than the quality of preprocessing and feature engineering.

5. Conclusion

This task provided a practical and conceptual understanding of three important classification algorithms applied to the Iris species dataset. SVM with Linear and RBF kernels delivered top accuracy by leveraging the class separability in the feature space. KNN proved to be a strong, intuitive baseline. Random Forest, the additional algorithm chosen for this task, matched the top classifiers in accuracy while also providing the added benefit of feature importance analysis.

The deeper takeaway from this study is that no single algorithm is universally best. The right choice depends on the data structure, size, dimensionality, and the specific requirements of the application — whether that is raw accuracy, interpretability, training speed, or scalability. Building familiarity with multiple algorithms and understanding their mechanics is essential to making sound model selection decisions in real-world machine learning projects.
